

1. Deep-Dive Mathematical Derivation from First Principles

In Nomura's Cash Equities Central Risk Book (CRB) department, you are responsible for managing the structural and residual risk of an automated execution inventory. Assuming asset factor exposures (β) are stationary across structural breaks can severely damage portfolio PnL.

During a liquidity shock, stock correlations converge to one, and style factors (e.g., Value vs. Momentum) experience sharp unwinds. A regime-switching linear factor model captures this by indexing the factor exposure vector to a discrete latent market state $s_t \in \{1, \dots, K\}$.

Let the equity return at time t be $r_t \in \mathbb{R}$. Let $\mathbf{f}_t \in \mathbb{R}^{d \times 1}$ represent the vector of d systemic risk factors (e.g., market excess return, size, liquidity factor scores). The model is formulated as:

$$r_t = \left(\beta^{(s_t)} \right)^\top \mathbf{f}_t + \epsilon_t$$

Where:

- s_t is the active discrete latent market regime state at time t .
- $\beta^{(s_t)} \in \mathbb{R}^{d \times 1}$ is the regime-conditional factor loading vector representing the conditional asset exposures.
- $\epsilon_t \sim \mathcal{N}\left(0, (\sigma^{(s_t)})^2\right)$ is the idiosyncratic residual noise term, whose variance is also conditional on the market state.

To derive the generalized parameter matrix equation, we aggregate the time-series over a historical estimation horizon $t = 1, \dots, T$. Let:

$$\mathbf{r} = \begin{bmatrix} r_1 \\ r_2 \\ \vdots \\ r_T \end{bmatrix} \in \mathbb{R}^{T \times 1}, \quad \mathbf{F} = \begin{bmatrix} \mathbf{f}_1^\top \\ \mathbf{f}_2^\top \\ \vdots \\ \mathbf{f}_T^\top \end{bmatrix} \in \mathbb{R}^{T \times d}, \quad \boldsymbol{\epsilon} = \begin{bmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_T \end{bmatrix} \in \mathbb{R}^{T \times 1}$$

Step 1: The Expectation-Maximization Objective for Regime GLS

When the exact state sequence is unobservable, we treat s_t probabilistically. Let $\gamma_t(s) = \mathbb{P}(s_t = s \mid \mathcal{F}_T, \theta)$ represent the smooth posterior responsibility coefficient generated during the Expectation (E) step of a hidden Markov process or mixture allocation engine.

The expected complete-data log-likelihood component $\mathcal{Q}_s(\beta^{(s)})$ isolated for a targeted state s is proportional to the responsibility-weighted log-density of a normal distribution:

$$\mathcal{Q}_s(\beta^{(s)}) = -\frac{1}{2} \sum_{t=1}^T \gamma_t(s) \left[\ln(2\pi) + \ln\left((\sigma^{(s)})^2\right) + \frac{\left(r_t - (\beta^{(s)})^\top \mathbf{f}_t\right)^2}{(\sigma^{(s)})^2} \right]$$

To optimize for the optimal factor loading vector $\hat{\beta}^{(s)}$, we drop the additive log-variance constant structural transformations that are independent of $\beta^{(s)}$, reducing the objective to a weighted residual sum of squares

problem:

$$\arg \min_{\boldsymbol{\beta}^{(s)}} \mathcal{J}_s \left(\boldsymbol{\beta}^{(s)} \right) = \sum_{t=1}^T \gamma_t(s) \left(r_t - \left(\boldsymbol{\beta}^{(s)} \right)^\top \mathbf{f}_t \right)^2$$

Step 2: Matrix Formulation of the Weighted Loss Operator

We construct a diagonal soft-regime filtering matrix $\mathbf{W}_s \in \mathbb{R}^{T \times T}$ using the calculated posterior probability arrays:

$$\mathbf{W}_s = \text{diag} \left(\gamma_1(s), \gamma_2(s), \dots, \gamma_T(s) \right) = \begin{bmatrix} \gamma_1(s) & 0 & \dots & 0 \\ 0 & \gamma_2(s) & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & \gamma_T(s) \end{bmatrix}$$

This allows us to rewrite the scalar summation objective function $\mathcal{J}_s \left(\boldsymbol{\beta}^{(s)} \right)$ as an inner-product matrix operation:

$$\mathcal{J}_s \left(\boldsymbol{\beta}^{(s)} \right) = \left(\mathbf{r} - \mathbf{F} \boldsymbol{\beta}^{(s)} \right)^\top \mathbf{W}_s \left(\mathbf{r} - \mathbf{F} \boldsymbol{\beta}^{(s)} \right)$$

Step 3: Expansion of the Matrix Form Vector Space

Expanding the transposed matrix terms:

$$\mathcal{J}_s \left(\boldsymbol{\beta}^{(s)} \right) = \left(\mathbf{r}^\top - \left(\boldsymbol{\beta}^{(s)} \right)^\top \mathbf{F}^\top \right) \mathbf{W}_s \left(\mathbf{r} - \mathbf{F} \boldsymbol{\beta}^{(s)} \right)$$

Distributing $\mathbf{W}_s$ across the internal matrix terms yields:

$$\mathcal{J}_s \left(\boldsymbol{\beta}^{(s)} \right) = \mathbf{r}^\top \mathbf{W}_s \mathbf{r} - \mathbf{r}^\top \mathbf{W}_s \mathbf{F} \boldsymbol{\beta}^{(s)} - \left(\boldsymbol{\beta}^{(s)} \right)^\top \mathbf{F}^\top \mathbf{W}_s \mathbf{r} + \left(\boldsymbol{\beta}^{(s)} \right)^\top \mathbf{F}^\top \mathbf{W}_s \mathbf{F} \boldsymbol{\beta}^{(s)}$$

Since $\mathbf{r}^\top \mathbf{W}_s \mathbf{F} \boldsymbol{\beta}^{(s)}$ is a scalar (1×1), it is equal to its transpose:

$$\left(\mathbf{r}^\top \mathbf{W}_s \mathbf{F} \boldsymbol{\beta}^{(s)} \right)^\top = \left(\boldsymbol{\beta}^{(s)} \right)^\top \mathbf{F}^\top \mathbf{W}_s^\top \mathbf{r}$$

Because $\mathbf{W}_s$ is a diagonal matrix, it is symmetric, meaning $\mathbf{W}_s^\top = \mathbf{W}_s$. This simplifies our expanded matrix expression to:

$$\mathcal{J}_s \left(\boldsymbol{\beta}^{(s)} \right) = \mathbf{r}^\top \mathbf{W}_s \mathbf{r} - 2 \left(\boldsymbol{\beta}^{(s)} \right)^\top \mathbf{F}^\top \mathbf{W}_s \mathbf{r} + \left(\boldsymbol{\beta}^{(s)} \right)^\top \mathbf{F}^\top \mathbf{W}_s \mathbf{F} \boldsymbol{\beta}^{(s)}$$

Step 4: First-Order Conditions and Optimization

To find the optimal parameter configurations, we compute the gradient of $\mathcal{J}_s$ with respect to the vector $\boldsymbol{\beta}^{(s)}$. We apply standard matrix calculus identities:

$$\frac{\partial (\mathbf{a}^\top \mathbf{x})}{\partial \mathbf{x}} = \mathbf{a} \quad \text{and} \quad \frac{\partial (\mathbf{x}^\top \mathbf{A} \mathbf{x})}{\partial \mathbf{x}} = 2\mathbf{A} \mathbf{x} \quad (\text{for symmetric } \mathbf{A})$$

Taking the partial derivative and setting it to the zero vector ($\mathbf{0}$):

$$\begin{aligned}\frac{\partial \mathcal{J}_s(\boldsymbol{\beta}^{(s)})}{\partial \boldsymbol{\beta}^{(s)}} &= \mathbf{0} - 2\mathbf{F}^\top \mathbf{W}_s \mathbf{r} + 2\mathbf{F}^\top \mathbf{W}_s \mathbf{F} \boldsymbol{\beta}^{(s)} = \mathbf{0} \\ -2\mathbf{F}^\top \mathbf{W}_s \mathbf{r} + 2\mathbf{F}^\top \mathbf{W}_s \mathbf{F} \boldsymbol{\beta}^{(s)} &= \mathbf{0} \\ 2\mathbf{F}^\top \mathbf{W}_s \mathbf{F} \boldsymbol{\beta}^{(s)} &= 2\mathbf{F}^\top \mathbf{W}_s \mathbf{r} \\ (\mathbf{F}^\top \mathbf{W}_s \mathbf{F}) \boldsymbol{\beta}^{(s)} &= \mathbf{F}^\top \mathbf{W}_s \mathbf{r}\end{aligned}$$

Assuming $(\mathbf{F}^\top \mathbf{W}_s \mathbf{F})$ is full rank and invertible, multiplying both sides by its inverse yields the final **Regime-Conditional GLS Estimator**:

$$\hat{\boldsymbol{\beta}}^{(s)} = (\mathbf{F}^\top \mathbf{W}_s \mathbf{F})^{-1} \mathbf{F}^\top \mathbf{W}_s \mathbf{r}$$

Phase B: The Pitfalls of Pooling Betas Across Regimes

Pooling data across regimes to run a single unconditioned Ordinary Least Squares (OLS) regression introduces structural specification bias. A pooled estimator enforces a static parameter assignment:

$$\hat{\boldsymbol{\beta}}_{\text{pooled}} = (\mathbf{F}^\top \mathbf{F})^{-1} \mathbf{F}^\top \mathbf{r}$$

Mathematically, as $T \rightarrow \infty$, the pooled matrix calculation converges to a probability-weighted blend of the true underlying structural parameter states:

$$\hat{\boldsymbol{\beta}}_{\text{pooled}} \xrightarrow{p} \sum_{s=1}^K \mathbb{P}(s_t = s) \cdot \hat{\boldsymbol{\beta}}^{(s)}$$

This regime-averaged estimate fails to accurately reflect any individual regime. For instance, consider a asset with a true market beta of $\beta^{(\text{Growth})} = 0.8$ during low-volatility regimes, which spikes to $\beta^{(\text{Crisis})} = 1.8$ during systemic liquidity drawdowns. If the historical frequency of crisis states is 10%, the pooled estimate is:

$$\hat{\beta}_{\text{pooled}} = (0.90 \times 0.8) + (0.10 \times 1.8) = 0.72 + 0.18 = 0.90$$

This introduces severe systematic risks to a Central Risk Book:

1. **Underestimation of Tail Risk:** During a market sell-off, the model expects the portfolio to respond with a beta of 0.90, when its true operational sensitivity is 1.8. This can result in unexpected tracking error and margin shortfalls.
 2. **Spurious Residual Alpha Identification:** Overestimating factor protections during low-volatility environments creates a false sense of security, masking structural risks within the book.
-

Phase C: The Label-Switching Identification Problem

In a mixture or regime-conditional specification, the log-likelihood function is invariant under permutations of the hidden state indexes. This is known as the **label-switching problem**.

Consider a 2-regime model with parameters $\theta = (\{\beta^{(1)}, \sigma^{(1)}\}, \{\beta^{(2)}, \sigma^{(2)}\})$. If we swap the labels such that state 1 becomes state 2 and state 2 becomes state 1, the calculated sequence likelihood remains identical:

$$\mathcal{L}(\mathbf{r} \mid \theta) = \mathcal{L}(\mathbf{r} \mid \theta_{\text{permuted}})$$

This lack of structural identification prevents unique global convergence. During cross-validation loops or rolling estimation windows, state 1 might randomly flip with state 2. This destabilizes the parameter series, rendering time-series tracking impossible.

The Solutions

- **Hard Operational Parameter Constraints:** To break likelihood symmetry, we enforce strict monotonic inequality parameters based on economic priors. For instance, we can order states by residual volatility thresholds:

$$\sigma^{(1)} < \sigma^{(2)} < \dots < \sigma^{(K)}$$

Alternatively, we can sort them by the conditional expected return of the market index factor: $\mu_1^{(\text{Market})} > \mu_2^{(\text{Market})}$.

- **A Priori Anchor Reference Initializations:** Rather than allowing the algorithm to initialize states symmetrically, we pre-classify historical periods using a deterministic macro proxy (e.g., periods where VIX > 30 are assigned to the crisis state). This seeds the optimization matrix with a clear structural definition from the first iteration.

2. Quantitative Infrastructure & Application of Theory

The following production-grade script provides an implementation of a Regime-Switching Factor Pricing Engine. It generates synthetic asset paths across structural state splits, calculates soft-regime weighted GLS projections, checks for cross-sectional pooling bias, enforces sorting constraints to resolve label-switching, and builds an interactive Plotly visualization framework.

```
import numpy as np
import pandas as pd
from scipy.stats import norm
import plotly.graph_objects as go
from plotly.subplots import make_subplots

# =====
# 1. GENERATE REGIME-CONDITIONAL RISK DATA
# =====
def generate_regime_factor_data(n_samples: int = 1200, seed: int = 42) -> tuple:
    """
    Generates asset returns driven by an underlying factor (e.g., Market Excess)
    where factor exposures (Betas) and residual noise vary by state.
    State 0: Quiet Growth (Low Beta, Low Residual Vol)
    State 1: Systemic Liquidity Shock (High Beta, High Residual Vol)
    """
```

```

np.random.seed(seed)

# Generate independent market factor tracking array
market_factor = np.random.normal(loc=0.0002, scale=0.010, size=n_samples)

# Define underlying hidden state trajectory (Markov Chain proxy)
# High persistence setup for economic state realities
latent_states = np.zeros(n_samples, dtype=int)
p_01, p_10 = 0.04, 0.12 # Transition likelihood parameters

for t in range(1, n_samples):
    if latent_states[t-1] == 0:
        latent_states[t] = np.random.choice(, p=[1 - p_01, p_01])
    else:
        latent_states[t] = np.random.choice(, p=[p_10, 1 - p_10])

# Structural Ground Truth Configuration Block
true_betas = {0: np.array([0.75]), 1: np.array([1.85])}
true_sigmas = {0: 0.004, 1: 0.018}

asset_returns = np.zeros(n_samples)
for t in range(n_samples):
    st = latent_states[t]
    beta_st = true_betas[st]
    sigma_st = true_sigmas[st]

    # Linear production matrix mapping equation
    asset_returns[t] = market_factor[t] * beta_st + np.random.normal(0,
sigma_st)

df = pd.DataFrame({
    'Market_Factor': market_factor,
    'Asset_Returns': asset_returns,
    'True_State': latent_states
})
return df, true_betas, true_sigmas

# =====
# 2. EXPECTATION-MAXIMIZATION MODEL EXTRACTION PIPELINE
# =====
class RegimeConditionalFactorModel:
    """
    Estimates regime-conditional betas via soft-weighted Generalized Least
    Squares.
    Solves the label-switching problem by enforcing variance ordering constraints:
    sigma_0 < sigma_1.
    """
    def __init__(self, n_regimes: int = 2, max_iter: int = 50):
        self.K = n_regimes
        self.max_iter = max_iter
        self.betas = {}
        self.sigmas = {}
        self.state_prior = None

```

```

def fit(self, X: np.ndarray, y: np.ndarray):
    T, d = X.shape

    # Seed parameters using a basic spatial split heuristic
    gamma = np.zeros((T, self.K))
    gamma[:, 0] = 0.60
    gamma[:, 1] = 0.40

    for iteration in range(self.max_iter):
        # -----
        # MAXIMIZATION STEP: ESTIMATE BETA VIA REGIME WEIGHTED GLS
        # -----
        for s in range(self.K):
            W_s = gamma[:, s]

            # Construct design matrices:  $X^T * W * X$ 
            XT_W = X.T * W_s
            XT_W_X = XT_W @ X
            XT_W_y = XT_W @ y

            # Compute analytical conditional matrix inversion solution
            beta_s = np.linalg.inv(XT_W_X) @ XT_W_y
            self.betas[s] = beta_s

            # Update residual conditional variance parameter
            residuals_s = y - X @ beta_s
            sigma_s = np.sqrt(np.sum(W_s * (residuals_s**2)) / np.sum(W_s))
            self.sigmas[s] = sigma_s

        # -----
        # RESOLVE LABEL SWITCHING VULNERABILITY
        # Enforce variance constraint ordering identity: State 0 must be lower
        variance

        # -----
        if self.sigmas > self.sigmas:
            # Swap parameter profiles across state maps
            self.betas, self.betas = self.betas, self.betas
            self.sigmas, self.sigmas = self.sigmas, self.sigmas
            gamma[:, :] = gamma[:, :]

        # -----
        # EXPECTATION STEP: COMPUTE RE-ALLOCATED RESPONSIBILITIES
        # -----
        self.state_prior = np.mean(gamma, axis=0)
        densities = np.zeros((T, self.K))

        for s in range(self.K):
            res = y - X @ self.betas[s]
            densities[:, s] = norm.pdf(res, loc=0, scale=self.sigmas[s])

        numerator = densities * self.state_prior
        denominator = np.sum(numerator, axis=1, keepdims=True)
        gamma = numerator / (denominator + 1e-12)

```

```

        return gamma

# =====
# 3. INTERACTIVE PLOTLY INTERVIEW DASHBOARD GENERATOR
# =====
def generate_regime_regression_dashboard(df: pd.DataFrame, gamma: np.ndarray,
pooled_beta: float, conditional_model):
    """
    Renders an interactive high-fidelity 2x2 validation matrix
    illustrating the operational implications of pooling bias vs. conditional GLS.
    """
    fig = make_subplots(
        rows=2, cols=2,
        subplot_titles=(
            "Cross-Sectional Scatter Profile & Regression Fits",
            "Latent Responsibility Timeline Distribution (Soft-Max Gamma)",
            "Pooled OLS Bias Spectrum vs. Regime Reality",
            "Residual Residual Variance Delta Profile"
        ),
        vertical_spacing=0.15, horizontal_spacing=0.12
    )

    # Panel 1: Cross-Section Scatter Map and Line Profiles
    fig.add_trace(go.Scatter(
        x=df['Market_Factor'], y=df['Asset_Returns'], mode='markers',
        marker=dict(color=gamma[:, 1], colorscale='Viridis', showscale=False,
size=5),
        name='Observed Returns'
    ), row=1, col=1)

    x_range = np.linspace(df['Market_Factor'].min(), df['Market_Factor'].max(),
100)

    # Superimpose structural lines
    fig.add_trace(go.Scatter(x=x_range, y=x_range * pooled_beta, mode='lines',
line=dict(color='crimson', width=3, dash='dash'), name='Pooled OLS Fit'), row=1,
col=1)
    fig.add_trace(go.Scatter(x=x_range, y=x_range * conditional_model.betas,
mode='lines', line=dict(color='emerald', width=3), name='Regime 0 (Quiet)'),
row=1, col=1)
    fig.add_trace(go.Scatter(x=x_range, y=x_range * conditional_model.betas,
mode='lines', line=dict(color='gold', width=3), name='Regime 1 (Crisis)'), row=1,
col=1)

    # Panel 2: Latent Responsibility Timeline distribution
    time_axis = np.arange(len(df))
    fig.add_trace(go.Scatter(x=time_axis, y=gamma[:, 1], mode='lines',
line=dict(color='cyan', width=1), name='P(Crisis Regime)'), row=1, col=2)

    # Panel 3: Pooled OLS Bias Spectrum Matrix Chart
    fig.add_trace(go.Bar(
        x=['True Quiet Beta', 'Pooled Estimate', 'True Crisis Beta'],
        y=[0.75, pooled_beta, 1.85],
        marker_color=['mediumseagreen', 'indianred', 'gold'],

```

```

        text=[f"0.75", f"{pooled_beta:.3f}", f"1.85"], textposition='auto'
    ), row=2, col=1)

    # Panel 4: Error Residual distribution under alternative states
    res_pooled = df['Asset>Returns'] - df['Market_Factor'] * pooled_beta
    fig.add_trace(go.Histogram(x=res_pooled, nbinsx=40, marker_color='orchid',
    opacity=0.6, name='Pooled Error Spread'), row=2, col=2)

    fig.update_layout(
        title_text="Nomura Cash Equities CRB – Regime Conditional Factor
    Performance Analyzer",
        title_x=0.5, height=900, width=1300, template="plotly_dark",
    showlegend=False
    )

    fig.update_xaxes(title_text="Systemic Factor Return Profile", row=1, col=1)
    fig.update_yaxes(title_text="Asset Return Profile", row=1, col=1)
    fig.update_xaxes(title_text="Historical Sequence Timeline Indexes", row=1,
    col=2)
    fig.update_yaxes(title_text="Calculated Model Allocation Weight (Gamma)",
    row=1, col=2)
    fig.update_yaxes(title_text="Calculated Beta Sensitivity Value", row=2, col=1)
    fig.update_xaxes(title_text="Residual Prediction Deviations", row=2, col=2)

    fig.show()

if __name__ == "__main__":
    # 1. Pipeline synthetic tracking series
    data_df, truth_betas, truth_sigmas = generate_regime_factor_data()

    X_matrix = data_df['Market_Factor'].values.reshape(-1, 1)
    y_vector = data_df['Asset>Returns'].values

    # 2. Extract standard naive pooled framework profile via ordinary matrix
    inverse
    pooled_b = np.linalg.inv(X_matrix.T @ X_matrix) @ X_matrix.T @ y_vector

    # 3. Fit Production Engine via structured Expectation Maximization loops
    model_engine = RegimeConditionalFactorModel(n_regimes=2, max_iter=40)
    calculated_gammas = model_engine.fit(X_matrix, y_vector)

    print("\n" + "="*60)
    print("REGIME CONDITIONAL PARAMETER CALIBRATION DETAILS:")
    print("="*60)
    print(f"Pooled Global OLS Regression Beta : {pooled_b:.4f}")
    print(f"Extracted Quiet Market State 0 Beta: {model_engine.betas:.4f} (True:
    0.75)")
    print(f"Extracted Crisis Market State 1 Beta: {model_engine.betas:.4f} (True:
    1.85)")
    print(f"Estimated Error Variance State 0 : {model_engine.sigmas:.5f}")
    print(f"Estimated Error Variance State 1 : {model_engine.sigmas:.5f}")
    print("="*60 + "\n")

    # 4. Render Validation Platform Canvas

```



```
generate_regime_regression_dashboard(data_df, calculated_gammas, pooled_b,
model_engine)
```

3. Strategic Interview Follow-Ups & Edge-Case Vulnerabilities

Follow-Up 1: Regularization in Regime Least Squares ($\hat{\beta}^{(s)}$)

Question: "Look at your derived equation: $\hat{\beta}^{(s)} = (\mathbf{F}^\top \mathbf{W}_s \mathbf{F})^{-1} \mathbf{F}^\top \mathbf{W}_s \mathbf{r}$. What structural risk arises if a specific market regime state s has a very low cumulative probability across the entire historical estimation horizon?"

- **The Answer:** If a regime s occurs rarely, the sum of its posterior weights is small ($\sum_{t=1}^T \gamma_t(s) \rightarrow 0$). This causes the diagonal matrix $\mathbf{W}_s$ to become highly sparse, with entry values approaching zero.
- As a result, the inner matrix product $(\mathbf{F}^\top \mathbf{W}_s \mathbf{F})$ becomes poorly conditioned or singular. Inverting this matrix magnifies numerical noise, leading to highly volatile, unreliable beta estimates $(\hat{\beta}^{(s)})$.
- **The Correction:** To resolve this in production, we add a ridge regularization penalty ($\lambda \mathbf{I}$) directly inside the conditional matrix inversion step:

$$\hat{\beta}_{\text{regularized}}^{(s)} = (\mathbf{F}^\top \mathbf{W}_s \mathbf{F} + \lambda \mathbf{I})^{-1} \mathbf{F}^\top \mathbf{W}_s \mathbf{r}$$

This regularizes the inversion process, shrinking the beta estimates toward zero when data in that regime is scarce.

Follow-Up 2: Endogeneity and Regime Transitions

Question: "Our factor model assumes the market factors $\mathbf{f}_t$ are completely exogenous to the error terms ϵ_t . However, if the Central Risk Book liquidates a large equity block, that execution can spike local volatility, triggering a transition into a crisis regime. How does this endogeneity distort your parameter estimation?"

- **The Trap:** Answering that increasing the sample size (T) will eliminate this bias. This is incorrect because endogeneity invalidates the core consistency assumptions of OLS and GLS.
- **The Answer:** When the book's liquidations impact the market, the error term ϵ_t (which captures execution slippage and microstructural noise) becomes correlated with the factor array $\mathbf{f}_t$ (market volatility and returns). This violates the exogeneity condition:

$$\mathbb{E}[\epsilon_t \mid \mathbf{f}_t] \neq 0$$

- This introduces endogeneity bias, causing the conditional GLS framework to overestimate the asset's true systematic beta during crisis periods. The model misattributes execution impact to external market factors rather than recognizing it as self-induced slippage.
- **The Correction:** To isolate self-induced impact from true market risk, we implement an **Instrumental Variables (IV) Two-Stage Least Squares (2SLS)** framework within the regime-switching engine. We use instruments $\mathbf{z}_t$ that are correlated with market liquidity but entirely independent of our internal

execution logs (such as foreign exchange volumes or global macroeconomic indicator updates) to clear the endogeneity from $\mathbf{F}$ before computing the conditional betas.